

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

EP-HelpDesk Sistem

Klijent: Elektroprivreda Bosne i Hercegovine

Naziv sistema: EP-HelpDesk

Verzija dokumenta: 1.0

Datum: April 2026

1. UVOD

1.1 Svrha sistema

Cilj projekta je razvoj centralizovanog, skalabilnog i inteligentnog **HelpDesk sistema** koji omogućava:

- efikasno upravljanje korisničkim zahtjevima (ticketing),
 - automatizovano rutiranje tiketa prema organizacionoj strukturi,
 - smanjenje opterećenja IT sektora kroz **self-service (knowledge base)**,
 - integraciju sa Microsoft ekosistemom (Active Directory, Office 365),
 - naprednu analitiku i praćenje performansi.
-

1.2 Opseg sistema

Sistem pokriva:

- sve zaposlenike kompanije kao korisnike (klijente),
 - sve organizacione jedinice (Direkcija, Podružnice, Poslovnice),
 - sve sektore i službe koje obrađuju zahtjeve,
 - centralizovanu administraciju i nadzor.
-

1.3 Definicije

- **Tiket (Ticket):** Zahtjev korisnika za podršku ili uslugu
 - **Organizaciona jedinica (OU):** Strukturalni element firme
 - **Handler grupa:** Tim koji obrađuje tikete
 - **Knowledge Base (KB):** Baza rješenja i članaka
 - **SSO:** Single Sign-On putem Active Directory
-

2. OPIS SISTEMA

2.1 Opšti pregled

EP-HelpDesk je **multi-tenant, hijerarhijski organizovan sistem** koji:

- koristi AD kao izvor identiteta,
 - automatski mapira korisnike na organizacione jedinice,
 - rutira tikete na osnovu:
 - lokacije (Company iz AD),
 - tipa servisa,
 - konfiguracije sistema.
-

2.2 Ključni ciljevi

- Smanjenje broja tiketa kroz KB ($\geq 30\%$)
 - Brža obrada zahtjeva (SLA optimizacija)
 - Transparentna komunikacija korisnik–administrator
 - Centralizovan nadzor nad cijelim sistemom
-

3. ORGANIZACIONA ARHITEKTURA

3.1 Hijerarhijski model

Nivo 1: Direkcija Društva

- Krovni nivo
- Globalna administracija
- Sektor za IKT → **SuperAdmin**

Nivo 2: Podružnice (Elektrodistribucije)

Primjeri:

- ED Sarajevo
- ED Zenica
- ED Tuzla

Svaka podružnica:

- ima svoje sektore i službe
 - upravlja svojim tiketima
-

Nivo 3: Poslovnice (gradovi)

- pripadaju podružnici
 - mogu imati:
 - vlastite službe
 - ili službe povezane sa sektorima podružnice
-

3.2 Pravila pripadnosti

- Korisnik pripada OU na osnovu AD atributa (Company, Department)
 - Tiket se inicijalno vezuje za OU korisnika
 - Sistem odlučuje routing na osnovu konfiguracije
-

3.3 Routing logika (kritično)

Tiket = (origin_unit + service_type)

→ mapiranje na:

- handler grupu (npr. IKT ED Zenica)

Podržano:

- manualno prosljeđivanje
 - automatska dodjela
-

4. FUNKCIONALNI ZAHTEVI

4.1 Upravljanje korisnicima

4.1.1 Autentikacija

- SSO putem Active Directory (Azure AD / Entra ID)
- Bez lokalnih lozinki

4.1.2 Povlačenje podataka iz AD-a

- ime i prezime
 - email
 - Company (ključni atribut)
 - Department
 - Manager (opcionally)
-

4.2 Upravljanje tiketima

4.2.1 Kreiranje tiketa (workflow)

1. Odabir servisa
 2. Odabir vrste zahtjeva
 3. Definisanje roka
 4. Unos opisa
-

4.2.2 Knowledge Base intercept (ključna funkcionalnost)

Prije submit-a:

- sistem vrši pretragu KB
- prikazuje slične probleme

Opcije:

- ✓ “Rješenje je pomoglo” → tiket se NE kreira
 - ✗ “Nije pomoglo” → kreiranje tiketa
-

4.2.3 Statusi tiketa

- Pending
 - Assigned
 - In Progress
 - Waiting for User
 - Resolved
 - Closed
-

4.2.4 Dodjela tiketa

Manualno:

- admin preuzima tiket

Automatski (opciono):

- **Least Busy algoritam**
 - ili Round Robin
-

4.2.5 Prosljeđivanje tiketa

Admin može:

- promijeniti grupu
 - eskalirati tiket
-

4.3 Time Tracking

- Start/Stop mehanizam
 - Automatski pause:
 - ako tab nije aktivan (> X min)
 - Sprečavanje:
 - beskonačnog vremena (infinity)
-

4.4 Komunikacija

4.4.1 Kanali

- in-app chat
- Edge integracija (notifikacije)

4.4.2 Funkcionalnosti

- razmjena poruka
 - zahtjev za dodatnim informacijama
 - sve se loguje (audit trail)
-

4.5 Remote Desktop (Quick Assist)

4.5.1 Funkcionalnost

Admin:

- šalje zahtjev za remote pristup

Korisnik:

- dobija notifikaciju
 - pokreće Quick Assist
-

4.5.2 Tehnički preduslovi

Potrebno:

- omogućen Quick Assist u Windows-u
- GPO konfiguracija:

- dozvoljen remote assistance
 - podrška za ms-quick-assist: protokol
-

4.6 Knowledge Base

4.6.1 Izvori

- zatvoreni tiketi
- ručno dodani članci

4.6.2 Funkcionalnosti

- pretraga (full-text + AI kasnije)
 - tagging
 - ocjenjivanje rješenja
-

4.7 Notifikacije

- Email (Office 365)
- Edge (push notifikacije)

Triggeri:

- novi tiket
 - dodjela
 - poruka
 - zatvaranje
-

4.8 Analitika i dashboard

4.8.1 KPI metrike

- broj tiketa po OU
 - prosječno vrijeme rješavanja
 - opterećenje admina
 - stopa rješavanja kroz KB
-

5. NEFUNKCIONALNI ZAHTJEVI

5.1 Performanse

- response time < 300ms

- podrška za ≥ 1000 simultanih korisnika
-

5.2 Skalabilnost

- horizontalno skaliranje backenda
 - modularna arhitektura
-

5.3 Sigurnost

- RBAC (role-based access control)
 - izolacija podataka po OU
 - audit log svih akcija
-

5.4 Dostupnost

- $\geq 99.9\%$ uptime
 - fallback za ključne servise
-

6. TEHNOLOŠKA ARHITEKTURA

6.1 Backend

- NestJS (TypeScript)
 - REST API + WebSocket (real-time)
-

6.2 Frontend

- React / Next.js
 - UI: ShadCN
-

6.3 Baza podataka

- MySQL
-

6.4 Autentikacija

- Microsoft Entra ID (Azure AD)
 - MSAL
-

6.5 Real-time komunikacija

- WebSockets (Socket.io)
-

7. INTEGRACIJE

7.1 Active Directory (ključna integracija)

Potrebno od IT odjela:

- Tenant ID
 - Client ID
 - Client Secret
 - API dozvole:
 - User.Read
 - Directory.Read.All
-

7.2 Office 365

- slanje email notifikacija
-

7.3 Edge integracija

- notifikacije kroz browser
 - potencijalna ekstenzija (nije obavezna u MVP)
-

8. ROLE I PRAVA

8.1 Tipovi korisnika

User (klijent)

- kreira tikete
- prati status

Agent (admin)

- obrađuje tikete
- komunicira sa korisnicima

Admin (lokalni)

- upravlja grupama i routingom

SuperAdmin (Direkcija IKT)

- globalni pristup
 - konfiguracija sistema
-

9. AUDIT I LOGGING

Sistem mora logovati:

- sve promjene statusa
 - komunikaciju
 - akcije admina
 - vrijeme rada
-

10. BUDUĆE NADOGRADNJE (nije MVP)

- AI klasifikacija tiketa
 - predikcija SLA
 - chatbot za self-service
 - mobilna aplikacija
-

ZAKLJUČAK

Ovaj dokument definiše:

- jasnu hijerarhiju organizacije
- precizan workflow tiketa
- ključne integracije (AD, O365, Quick Assist)
- skalabilnu enterprise arhitekturu

SRS UPDATE v1.1 – EP-HelpDesk

Izmjene u odnosu na v1.0

- 7.3 Edge ekstenzija → uključena kao core komponenta sistema
 - 10. Buduće nadogradnje → detaljno definisane (bez mobilne aplikacije)
-

7.3 Edge Ekstenzija (OBAVEZNA KOMPONENTA)

7.3.1 Uloga u sistemu

Edge ekstenzija predstavlja **client-side agent sloj** koji omogućava:

- real-time komunikaciju
 - sistemske notifikacije
 - integraciju sa Windows okruženjem
 - iniciranje remote sesija
-

7.3.2 Funkcionalnosti

1. Autentikacija

- koristi postojeći SSO (Azure AD session)
 - silent login kroz browser context
-

2. Notifikacije (kritično)

- toast notifikacije u Windows/Edge
 - triggeri:
 - nova poruka
 - zahtjev za remote pristup
 - promjena statusa tiketa
-

3. In-App Chat (lightweight client)

- prikaz poruka bez otvaranja web aplikacije
 - quick reply UI
 - sinhronizacija sa backendom (WebSocket)
-

4. Remote Desktop inicijacija

Admin akcija:

- klik na "Request Remote"

Ekstenzija:

- prima event
 - pokreće:
 - ms-quick-assist: protokol
-

5. Background Listener

- WebSocket konekcija
 - fallback na polling (ako WS padne)
-

7.3.3 Tehnička arhitektura

Komponente:

- background script (listener)
 - content script (UI injection opcionalno)
 - popup UI (mini dashboard)
-

7.3.4 Sigurnost

- token-based autentikacija (MSAL)
 - nema lokalnog storage osjetljivih podataka
 - svi requesti idu preko HTTPS
-

10. BUDUĆE NADOGRADNJE (DETALJNO)

10.1 AI Knowledge Base (Faza 2)

Funkcionalnosti:

- semantička pretraga (vector search)
- embedding model (OpenAI / lokalni model kasnije)
- automatsko predlaganje rješenja

Napredno:

- "similar tickets clustering"
- auto-tagging tiketa

10.2 AI Ticket Classification

- automatsko prepoznavanje:
 - servisa
 - prioriteta
 - handler grupe

Koristi:

- NLP model nad opisom tiketa
-

10.3 SLA Management System

Funkcionalnosti:

- definisanje SLA po:
 - servisu
 - OU
- automatski timeri:
 - response time
 - resolution time

Eskalacije:

- upozorenja
 - auto-eskalacija
-

10.4 Advanced Routing Engine

Upgrade sa:

- basic mapping

Na:

- rule-based engine:
 - IF (OU + Service + Priority)
 - THEN (Group + SLA + Priority override)
-

10.5 Workforce Optimization

- workload balancing

- predikcija opterećenja
 - preporuke za raspodjelu tiketa
-

10.6 Audit & Compliance Module

- kompletan audit trail
 - export logova
 - compliance-ready (ISO-like)
-

10.7 Reporting Engine

- custom report builder
 - export:
 - PDF
 - Excel
-

10.8 Notification Engine (Advanced)

- templating system
 - rule-based notifikacije
 - multi-channel:
 - email
 - Edge
 - (kasnije Teams opcionalno)
-

10.9 Integration Layer (API-first)

- REST + Webhook support
 - buduće integracije:
 - ERP
 - HR sistemi
-

10.10 Incident & Problem Management (ITIL layer)

- povezivanje tiketa u incidente
- root cause analysis

- problem records

REVIDIRANI ZAKLJUČAK

Sistem sada uključuje:

- punu **client-side integraciju (Edge ekstenzija)**
- definisanu **AI strategiju**
- jasan put ka **enterprise ITIL-aligned platformi**

DATA MODEL – KONCEPTUALNI DIZAJN (ERD LOGIKA)

1. CORE DOMENE

Sistem se dijeli na 6 glavnih domena:

1. Organizational Structure

- OrganizationalUnit (Direkcija, Podružnica, Poslovnica, Sektor, Služba)

2. Identity & Access

- User
- Role
- Group (handler timovi)

3. Ticketing System

- Ticket
- TicketActivity (audit + chat)
- TicketAssignment
- TicketTimeLog

4. Knowledge Base

- KnowledgeArticle
- KnowledgeFeedback

5. Routing & Configuration

- Service
- ServiceCategory
- RoutingRule

6. System & Audit

- Notification
- AuditLog

2. KLJUČNA ARHITEKTONSKA ODLUKA (KRITIČNO)

👉 **OrganizationalUnit = self-referencing tree**

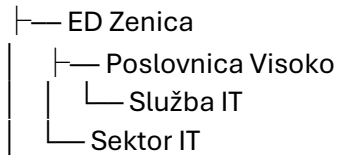
Jedna tabela pokriva:

- Direkciju
- Podružnice
- Poslovnice

- Sektore
- Službe

Struktura:

Direkcija



3. PRISMA SCHEMA (PRODUCTION READY)

3.1 Organizational Structure

```
model OrganizationalUnit {
  id      String @id @default(uuid())
  name    String
  type    OrganizationalUnitType
  parentId String?
  parent  OrganizationalUnit? @relation("OUHierarchy", fields: [parentId], references: [id])
  children OrganizationalUnit[] @relation("OUHierarchy")

  // Metadata
  code    String? @unique
  isActive Boolean @default(true)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  users    User[]
  groups   Group[]
}

enum OrganizationalUnitType {
  DIRECTORATE
  BRANCH
  OFFICE
  SECTOR
  SERVICE
}
```

3.2 Users & Roles

```
model User {
  id      String @id @default(uuid())
```



```

email    String @unique
fullName String

adObjectId String @unique // Azure AD ID
company  String?
department String?

organizationalUnitId String
organizationalUnit OrganizationalUnit @relation(fields: [organizationalUnitId], references:
[id])

roleId    String
role      Role @relation(fields: [roleId], references: [id])

groups    GroupMember[]

isActive  Boolean @default(true)

createdAt DateTime @default(now())
}

```

```

model Role {
  id    String @id @default(uuid())
  name  String @unique

  users User[]
}

```

3.3 Groups (Handler Teams)

```

model Group {
  id    String @id @default(uuid())
  name  String

  organizationalUnitId String
  organizationalUnit OrganizationalUnit @relation(fields: [organizationalUnitId], references:
[id])

  members    GroupMember[]

  isActive  Boolean @default(true)

  createdAt DateTime @default(now())
}

```

```

model GroupMember {
  id    String @id @default(uuid())

  userId String
  groupId String

  user  User @relation(fields: [userId], references: [id])
  group Group @relation(fields: [groupId], references: [id])

  @@unique([userId, groupId])
}

```

3.4 Ticketing Core

```

model Ticket {
  id      String @id @default(uuid())
  title   String
  description String

  status   TicketStatus
  priority TicketPriority

  requesterId String
  requester  User @relation("Requester", fields: [requesterId], references: [id])

  assignedGroupId String?
  assignedGroup  Group? @relation(fields: [assignedGroupId], references: [id])

  assignedUserId String?
  assignedUser   User? @relation("Assignee", fields: [assignedUserId], references: [id])

  serviceId String
  service   Service @relation(fields: [serviceId], references: [id])

  originUnitId String
  originUnit  OrganizationalUnit @relation(fields: [originUnitId], references: [id])

  dueDate  DateTime?

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  activities TicketActivity[]
  timeLogs   TicketTimeLog[]
}

```

```
enum TicketStatus {  
    PENDING  
    ASSIGNED  
    IN_PROGRESS  
    WAITING_FOR_USER  
    RESOLVED  
    CLOSED  
}
```

```
enum TicketPriority {  
    LOW  
    MEDIUM  
    HIGH  
    CRITICAL  
}
```

3.5 Ticket Activity (Chat + Audit)

```
model TicketActivity {  
    id    String @id @default(uuid())  
  
    ticketId String  
    ticket  Ticket @relation(fields: [ticketId], references: [id])  
  
    userId String  
    user   User @relation(fields: [userId], references: [id])  
  
    type  ActivityType  
    content String  
  
    createdAt DateTime @default(now())  
}
```

```
enum ActivityType {  
    COMMENT  
    STATUS_CHANGE  
    ASSIGNMENT  
    SYSTEM  
}
```

3.6 Time Tracking

```
model TicketTimeLog {  
    id    String @id @default(uuid())
```

```

ticketId String
ticket Ticket @relation(fields: [ticketId], references: [id])

userId String
user User @relation(fields: [userId], references: [id])

startedAt DateTime
endedAt DateTime?

duration Int? // seconds

createdAt DateTime @default(now())
}

```

3.7 Knowledge Base

```

model KnowledgeArticle {
  id String @id @default(uuid())
  title String
  content String

  serviceId String?
  service Service? @relation(fields: [serviceId], references: [id])

  createdById String
  createdBy User @relation(fields: [createdById], references: [id])

  isPublished Boolean @default(false)

  createdAt DateTime @default(now())
}

```

```

model KnowledgeFeedback {
  id String @id @default(uuid())

  articleId String
  article KnowledgeArticle @relation(fields: [articleId], references: [id])

  userId String
  user User @relation(fields: [userId], references: [id])

  isHelpful Boolean

  createdAt DateTime @default(now())
}

```

3.8 Services & Routing

```
model Service {
  id      String @id @default(uuid())
  name    String

  categoryId String?
  category  ServiceCategory? @relation(fields: [categoryId], references: [id])

  isActive Boolean @default(true)
}
```

```
model ServiceCategory {
  id  String @id @default(uuid())
  name String

  services Service[]
}
```

```
model RoutingRule {
  id      String @id @default(uuid())

  serviceId String
  service  Service @relation(fields: [serviceId], references: [id])

  unitId String
  unit   OrganizationalUnit @relation(fields: [unitId], references: [id])

  groupId String
  group  Group @relation(fields: [groupId], references: [id])
}
```

3.9 Notifications

```
model Notification {
  id      String @id @default(uuid())

  userId String
  user   User @relation(fields: [userId], references: [id])

  title String
  content String

  isRead Boolean @default(false)
```

```
    createdAt DateTime @default(now())  
}
```

3.10 Audit Log

```
model AuditLog {  
  id    String @id @default(uuid())  
  
  userId  String?  
  action  String  
  entity  String  
  entityId String  
  
  metadata Json?  
  
  createdAt DateTime @default(now())  
}
```

4. KLJUČNE PREDNOSTI OVOG MODELA

- ✓ Jedinstvena OU hijerarhija (fleksibilna)
- ✓ Čist separation of concerns
- ✓ Spreman za:
 - AI nadogradnje
 - SLA sistem
 - napredni routing
- ✓ Skalabilan (enterprise ready)

SYSTEM ARCHITECTURE – EP-HelpDesk

1. ARHITEKTONSKI PRINCIPI

Sistem će biti zasnovan na:

✓ **Modular Monolith (Phase 1)**

- jedan deployable servis
 - jasno odvojeni moduli
 - spremno za kasniji prelazak na microservices
-

✓ **Domain-Driven Design (DDD – light verzija)**

- svaki modul = business domen
 - minimalno curenje logike između modula
-

✓ **Event-Driven komunikacija (interno)**

- koristi se za:
 - notifikacije
 - audit log
 - real-time update
-

✓ **Clean Architecture (layered)**

Svaki modul ima:

- Controller (API layer)
 - Service (business logic)
 - Repository (Prisma)
 - DTO (validation)
 - Events
-

2. HIGH-LEVEL ARHITEKTURA

[React / Next.js]

↓

[API Gateway Layer (NestJS Controllers)]

↓

[Domain Modules]

↓
[Prisma ORM]
↓
[MySQL]
Parallel:
[WebSocket Gateway]
↕
[Event Bus]

3. NESTJS MODULE BREAKDOWN (KRITIČNO)

CORE MODULI

3.1 AuthModule

Odgovornost:

- integracija sa Azure AD (MSAL)
- validacija JWT tokena
- ekstrakcija user podataka

Komponente:

- AuthGuard
 - CurrentUser decorator
 - AuthService
-

3.2 UserModule

Odgovornost:

- sync korisnika iz AD-a
 - mapiranje na OrganizationalUnit
 - role management
-

3.3 OrganizationalUnitModule

Odgovornost:

- upravljanje hijerarhijom
- tree operacije (parent/child)
- lookup za routing

3.4 GroupModule

Odgovornost:

- handler grupe
- članstvo
- workload informacije

3.5 TicketModule (CORE SISTEMA)

Odgovornost:

- lifecycle tiketa
- kreiranje
- statusi
- assignment

Interna struktura:

ticket/

- └─ ticket.controller.ts
- └─ ticket.service.ts
- └─ ticket.repository.ts
- └─ ticket-routing.service.ts
- └─ ticket-assignment.service.ts
- └─ ticket.events.ts

Sub-servisi:

TicketRoutingService

- određuje groupId
- koristi:
 - RoutingRule
 - originUnit
 - service

TicketAssignmentService

- auto-assign:

- least busy
 - round robin
-
-

3.6 TicketActivityModule

Odgovornost:

- chat
 - audit trail
 - status promjene
-

3.7 TimeTrackingModule

Odgovornost:

- start/stop
 - validacija trajanja
 - anti-abuse logika
-

3.8 KnowledgeBaseModule

Odgovornost:

- članci
 - pretraga
 - povezivanje sa tiketima
-

3.9 NotificationModule

Odgovornost:

- centralni notification engine
-

Kanali:

- Email
 - WebSocket
 - Edge Extension
-

3.10 WebSocketModule

Odgovornost:

- real-time komunikacija
 - event emit
-

Gateway:

- ticket-updates
 - chat-messages
 - notifications
-
-

3.11 AuditModule

Odgovornost:

- centralizovan audit log
 - zapis svih akcija
-
-

3.12 ServiceModule

Odgovornost:

- definicija servisa
 - kategorije
-

3.13 RoutingModule

Odgovornost:

- upravljanje routing pravilima
 - konfiguracija admina
-

4. KOMUNIKACIJA IZMEĐU MODULA

Pattern: Event-based

Primjer:

 Kreiranje tiketa:

1. TicketService.create()

2. emituje event:

TicketCreatedEvent

Reaguju:

- NotificationModule → šalje email
 - WebSocket → push update
 - AuditModule → loguje akciju
-
-

Promjena statusa:

Event:

TicketStatusChangedEvent

5. WEBSOCKET ARHITEKTURA

Gateway:

```
@WebSocketGateway()  
class TicketGateway {}
```

Kanali:

- user:{userId}
 - group:{groupId}
 - ticket:{ticketId}
-

Flow:

Admin napiše poruku
→ TicketActivityService
→ emit event
→ WebSocketGateway
→ Edge Extension / Web App

6. EDGE EXTENSION FLOW

Real-time:

Backend (WS)

- Extension background script
 - prikaz notifikacije
-

Remote:

Admin klik

- emit event
 - extension
 - pokreće ms-quick-assist
-

7. TIME TRACKING LOGIKA (ANTI-ABUSE)

Problem:

- admin ostavi otvoren tab

Rješenje:

- heartbeat sistem (frontend)
 - ako nema aktivnosti:
 - backend zatvara session
-

8. SECURITY MODEL

RBAC

Roles:

- USER
 - AGENT
 - ADMIN
 - SUPER_ADMIN
-

Guardovi:

- AuthGuard
 - RoleGuard
 - OUAccessGuard
-

Pravilo:

- admin vidi samo svoju OU (osim super admin)

9. TRANSACTION STRATEGIJA

Koristi se Prisma:

Primjeri:

- create ticket + activity
- assign + log

10. ERROR HANDLING

Global:

- ExceptionFilter

Standard:

```
{
  "code": "TICKET_NOT_FOUND",
  "message": "...",
  "details": {}
}
```

11. FOLDER STRUKTURA (PRODUCTION READY)

src/

```
|— modules/
|   |— auth/
|   |— users/
|   |— organizational-unit/
|   |— group/
|   |— ticket/
|   |— ticket-activity/
|   |— time-tracking/
|   |— knowledge-base/
|   |— notification/
|   |— websocket/
|   |— audit/
|   |— service/
|   |— routing/
|
|— common/
|   |— guards/
|   |— decorators/
|   |— filters/
|   |— interceptors/
```

```
|  
|— prisma/  
|  
|— config/
```

12. KLJUČNI TOKOVI (END-TO-END)

FLOW: Kreiranje tiketa

1. User → API
 2. KB check
 3. TicketService.create()
 4. RoutingService
 5. AssignmentService
 6. emit event
 7. Notification + WS
-

FLOW: Chat

1. Admin → poruka
 2. TicketActivityService
 3. emit event
 4. WebSocket → user
-

FLOW: Remote request

1. Admin klik
 2. emit event
 3. extension hvata
 4. pokreće Quick Assist
-

13. ZAŠTO JE OVO DOBRO

- ✓ Jasno modularno
- ✓ Skalabilno
- ✓ Cursor-friendly
- ✓ Spremno za AI upgrade
- ✓ Enterprise standard

ULTIMATE CURSOR SETUP – EP-HelpDesk

1. MASTER PROMPT (ULAZ ZA CURSOR)

Ovo je **prva stvar koju ubacuješ u Cursor (kao početni prompt)**.

Writing

You are a senior software engineer working on an enterprise-grade HelpDesk system for a large organization with complex hierarchical structure.

Your job is NOT just to write code, but to:

- design scalable architecture
- enforce consistency
- prevent bad patterns
- think in systems, not files

PROJECT CONTEXT

This is a centralized HelpDesk system with:

- Organizational hierarchy:
 - Directorate (root)
 - Branches (ED Sarajevo, ED Zenica...)
 - Offices (city-level)
 - Sectors and Services
- Users belong to Organizational Units (OU)
- Tickets must be routed based on:
 - user's OU
 - selected service
 - routing rules
- The system uses:
 - NestJS (backend)
 - Prisma + MySQL
 - React (frontend)
 - Azure AD (SSO via MSAL)
 - WebSockets (real-time)
 - Edge Extension (notifications + remote actions)

CORE PRINCIPLES (MANDATORY)

1. NEVER mix responsibilities between modules
2. ALWAYS follow modular architecture (NestJS modules)
3. ALWAYS use DTOs with validation
4. NEVER access Prisma directly from controllers
5. ALL business logic must go through services
6. ALWAYS emit events for side effects (notifications, audit)
7. NEVER hardcode routing logic – use database-driven rules
8. ALWAYS think about multi-tenant isolation (OU-based access)

TICKET SYSTEM RULES

- Ticket MUST have:
 - originUnitId
 - serviceId
 - requesterId
- Ticket creation flow:
 1. validate input
 2. run knowledge base check
 3. create ticket
 4. resolve routing (RoutingService)
 5. assign group/user (AssignmentService)
 6. emit TicketCreatedEvent

ORGANIZATIONAL RULES

- OrganizationalUnit is a TREE (self-referencing)
- NEVER hardcode hierarchy
- ALWAYS query via parent/child relationships

SECURITY RULES

- Use Guards:

- AuthGuard
 - RoleGuard
 - OUAccessGuard
 - Admins can ONLY see their OU data
 - SuperAdmin can see everything
-

REAL-TIME RULES

- ALL chat/messages go through WebSocket
 - ALWAYS emit events → WebSocket Gateway handles delivery
-

EDGE EXTENSION RULES

- Treat extension as a client
 - Send events via WebSocket
 - NEVER create separate logic for extension
-

TIME TRACKING RULES

- NEVER trust frontend time
 - ALWAYS validate on backend
 - prevent infinite sessions
-

CODE STYLE

- Use TypeScript strictly
 - Use clear naming (no abbreviations)
 - Prefer small functions
 - Avoid magic values
 - Always write readable, production-ready code
-

YOUR BEHAVIOR

Before writing code:

1. Analyze requirements
2. Identify module

3. Define data flow
4. Then implement

If something is unclear:

- ASK for clarification
- DO NOT guess

You are building a production enterprise system.
Act accordingly.

2. .CURSORRULES STRUKTURA (KRITIČNO)

Napraviti folder:

.cursor/
rules/

3. GLAVNI RULE FAJL

.cursor/rules/project.mdc

Writing

EP-HelpDesk – Global Rules

Architecture

- Use NestJS modular architecture
- Each domain must be isolated
- No cross-module direct DB access

Database

- Use Prisma ORM only
- Never write raw SQL unless explicitly required
- Always use relations properly

Organizational Model

- Always use OrganizationalUnit tree
- Never hardcode OU logic

Ticketing

- Always use services for:
 - routing
 - assignment

- Never assign tickets manually in controller

Events

- All side effects must go through events

Naming

- Use full descriptive names
- No abbreviations

Code Quality

- Must be production-ready
- No hacks
- No temporary solutions

4. BACKEND RULES

.cursor/rules/backend.mdc

Writing

Backend Rules (NestJS)

Structure

Each module must contain:

- controller
- service
- dto
- repository (optional)
- events

Controllers

- Only handle HTTP
- No business logic

Services

- Contain ALL business logic

DTOs

- Use class-validator
- Always validate input

Prisma

- Access ONLY from service or repository layer

Events

- Use event-driven pattern for:
 - notifications
 - audit logs
 - websocket updates

Guards

- Always protect endpoints
-

5. DATABASE RULES

.cursor/rules/database.mdc

Writing

Database Rules

Prisma

- Use schema as source of truth
- Always define relations

Migrations

- Never modify DB manually
- Always use migrations

Constraints

- Use foreign keys
- Use unique constraints

Performance

- Avoid N+1 queries
 - Use include/select properly
-

6. FRONTEND RULES

.cursor/rules/frontend.mdc

Writing

Frontend Rules

Architecture

- Use modular structure
- Separate:
 - pages
 - components
 - services

API

- Use centralized API client

State

- Avoid global chaos
- Use structured state management

UX

- Always show:
 - loading states
 - error states

Forms

- Validate before submit

7. WEBSOCKET RULES

.cursor/rules/websocket.mdc

Writing

WebSocket Rules

Communication

- Use event-based communication

Channels

- user:{userId}
- group:{groupId}
- ticket:{ticketId}

Behavior

- Never send data directly from services
- Always go through gateway

Sync

- Keep frontend and extension in sync

8. AI DEVELOPMENT WORKFLOW (NAJBITNIJI DIO)

Ovo je kako koristiš Cursor:

✅ FAZA 1 – Scaffold

Prompt:

Create NestJS module for Ticket with controller, service, DTOs and basic CRUD structure following project rules.

✅ FAZA 2 – Business logic

Implement ticket creation flow with routing and assignment using services and events.

✅ FAZA 3 – Integracije

Add WebSocket events for ticket updates and chat messages.

✅ FAZA 4 – Hardening

Add validation, guards, and error handling to ticket module.

9. ANTI-PATTERN ZA CURSOR (KRITIČNO)

Nikad mu ne govoriti:

- ✗ “napravi brzo”
- ✗ “samo da radi”
- ✗ “preskoči ovo”

Uvijek:

- ✓ “production-ready”
- ✓ “follow rules”
- ✓ “respect architecture”

10. REALNOST (ISKRENO)

Sa ovim setupom:

👉 Cursor će raditi kao **mid-senior developer**

👉 Bez ovoga → radi kao junior koji halucinira

ZAKLJUČAK

Sada imaš:

- ✓ Enterprise SRS
- ✓ Data model
- ✓ System architecture
- ✓ AI control sistem (.cursorrules + master prompt)